# AI FILE MANAGEMENT SYSTEM

**MINI PROJECT REPORT**

***Submitted by***

**BEN SEBASTIAN JOSEPH**

(AJC23AD017)

***in partial fulfillment for the award of***

***the degree of***

**BACHELOR OF TECHNOLOGY**

***in***

ARIFICIAL INTELLIGENCE AND

DATA SCIENCE

**AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS),**

**KANJIRAPALLY**

APRIL 2026

**DEPARTMENT OF ARITIFICIAL INTELLIGENCE AND INFORMATION TECHNOLOGY**

**AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS), KANJIRAPALLY**

# DECLARATION

I undersigned hereby declare that the project report "**AI FILE MANAGEMENT SYSTEM**", submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Ms. SELIN SAM**. This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources. We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

BEN SEBASTIAN JOSEPH

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND INFORMATION TECHNOLOGY**

**AMAL JYOTHI COLLEGE OF ENGINEERING (AUTONOMOUS), KANJIRAPALLY**

# BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**AI FILE MANAGEMENT SYSTEM**" is a bonafide record of the mini project work carried out by **BEN SEBASTIAN JOSEPH(AJC23AD017),** during the academic year 2025 -2026 in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence and Data Science by APJ Abdul Kalam Technological University.

**Ms. SELIN SAM**
**Assistant Professor**
**Project Guide**

**Dr. MANOJ T JOY**
**Head of the Department**

Submitted for Mini-Project Internal examination held on _____

**INTERNAL EXAMINER**

# ACKNOWLEDGEMENT

First of all, we submit all glory and honour to **GOD ALMIGHTY** for his eternal love and protection throughout the project. The tremendous amount of help that we received from a number of people gave us the incentive to complete this project to the best of our capability.

I express my sincere thanks to **Rev. Dr. Roy Abraham**, Manager of our college for giving an opportunity to undertake this project. I am very much grateful to **Dr. Lillykutty Jacob**, Principal of our college for supporting all the way long. I express my special gratitude to the HOD of Artificial Intelligence and Information Technology **Dr. Manoj T Joy**, for providing me constant guidance and encouragement throughout the project.

I also express my gratitude to the Project Coordinators, **Ms Selin Sam**, Assistant Professor for her inspiration and suggestions.

I feel deeply honored in expressing sincere thanks to my guide **Ms. SELIN SAM**, for his/her guidance and support in completing the project. I have to appreciate the guidance
given by the panel members during the project presentation, thanks to their comments and advice.

Last but not the least, I place a deep sense of gratitude to my family members and my friends who have been constant source of inspiration during the preparation of this project work.

# ABSTRACT

The AI File Management System is designed to revolutionize how individuals and organizations manage, search, and extract value from their digital documents. Traditional file management systems rely heavily on exact keyword matches and manual folder organization, which becomes inefficient as data volume grows. This project introduces an intelligent platform that leverages Artificial Intelligence to deliver advanced features such as semantic search, automated document summarization, Optical Character Recognition (OCR), smart tagging, and content deduplication. Built with a FastAPI backend and an interactive Streamlit frontend, the system processes uploaded files, extracts textual content, and generates vector embeddings stored in a Qdrant database. This architecture enables users to perform context-aware natural language searches, instantly summarize large documents, and maintain an organized, duplicate-free repository, ultimately enhancing productivity and information retrieval.

# CONTENTS


| TITLE | PAGE NO |
|---|---|
| **1. INTRODUCTION** | 8 - 9 |
| 1.1 General Background | 8 |
| 1.2 Motivation | 8 |
| 1.3 Overview of the report | 8 |
| 1.4 Proposed system | 9 |
| 1.5 Objectives | 9 |
| 1.6 Significance of the system | 9 |
| **2. EXISTING SYSTEM** | 10 |
| 2.1 Drawbacks | 10 |
| **3. SYSTEM REQUIREMENTS AND DESIGN OVERVIEW** | 11 - 14 |
| 3.1 Functional Requirements | 11 |
| 3.2 System Design | 12 - 14 |
| 3.2.1. Class Diagram | 12 |
| 3.2.2. Use Case Diagram | 13 |
| 3.2.3. Database Design | 14 |
| **4. IMPLEMENTATION** | 15 - 16 |
| 4.1 Proposed System | 15 |
| 4.2 Modules in Project | 16 |
| **5. TESTING AND RESULTS** | 17 - 20 |
| 5.1 Validation | 17 |
| 5.2 Results | 18 - 20 |
| **6. CONCLUSION** | 21 |
| **REFERENCES** | 22 - 25 |
| **APPENDIX - A: Source Code** | 22 - 25 |
| **APPENDIX - B: User Manual** | 25 |

# LIST OF FIGURES


| No | Name | Page no |
|---|---|---|
| 1 | Fig. 1 (System Dashboard / Overview Page) | 18 |
| 2 | Fig. 2 (File Upload and Ingestion Interface) | 18 |
| 3 | Fig. 3 (Document Library / File Listing View) | 19 |
| 4 | Fig. 4 (Semantic Search Interface and Results) | 19 |
| 5 | Fig. 5 (AI Document Summarization View) | 20 |
| 6 | Fig. 6 (OCR Text Extraction Results for Scanned Images) | 20 |
| 7 | Fig. 8 (Class Diagram) | 12 |
| 8 | Fig. 9 (Use Case Diagram) | 13 |
| 9 | Fig. 10 (Database Design Diagram) | 14 |

# CHAPTER 1

# INTRODUCTION

## 1.1 GENERAL BACKGROUND

In the modern digital era, individuals and enterprises produce and store massive quantities of documents, images, and files daily. As the volume of unstructured data grows exponentially, traditional directory-based file systems struggle to provide efficient retrieval and management. Users spend a significant amount of time manually sorting files and searching for specific information hidden deep within text documents or scanned images.

## 1.2 MOTIVATION

The primary motivation behind this project is to eliminate the inefficiencies associated with manual file management. By integrating Artificial Intelligence components like Natural Language Processing (NLP) and Computer Vision (OCR) into a unified file storage system, we can automate tedious tasks such as tagging, summarizing, and reading scanned documents, allowing users to converse with and search their data intelligently.

## 1.3 OVERVIEW OF THE REPORT

This report details the development of the AI File Management System. Chapter 2 discusses the drawbacks of existing traditional systems. Chapter 3 outlines the system requirements and architectural design. Chapter 4 provides an in-depth look at the technology stack and implementation of core modules. Chapter 5 covers testing methodologies and visual results, ending with the conclusion in Chapter 6.

## 1.4 PROPOSED SYSTEM

The proposed system is a modern web application featuring an intuitive Streamlit frontend and a highly performant FastAPI backend. It incorporates an AI processing pipeline capable of executing OCR, generating automated text summaries, generating vector embeddings, and executing semantic search queries using the Qdrant Vector Database.

## 1.5 OBJECTIVES

- To develop a seamless file upload and listing module.
- To implement a semantic search engine capable of understanding the context of user queries rather than just keyword matching.
- To integrate an AI-powered summarization tool for rapid document comprehension.
- To utilize OCR technology to extract text from image-based files.
- To implement algorithms that detect and prevent duplicate file uploads, saving storage space.

## 1.6 SIGNIFICANCE OF THE SYSTEM

This project bridges the gap between basic cloud storage and intelligent data processing. It significantly reduces the time required to retrieve information, enhances data accessibility for visually heavy documents via OCR, and drastically improves overall digital workspace organization.

# CHAPTER 2

# EXISTING SYSTEM

## 2.1 DRAWBACKS OF TRADITIONAL FILE MANAGEMENT METHODS

- **Keyword Dependency:** Traditional search bars (like Windows Explorer or basic cloud storage) rely strictly on exact word matches. If a user searches for "automobile" but the document uses the word "car," the file will not be found.
- **Lack of Content Understanding:** Existing systems cannot provide a quick synopsis of a file; the user must manually open and read the document to understand its contents.
- **Inaccessibility of Scanned Documents:** Traditional file systems treat images and scanned PDFs purely as visual data, making the text within them completely unsearchable.
- **Storage Inefficiency:** Users frequently download and upload the same files multiple times. Traditional systems often fail to recognize duplicate content if the file name is slightly different, leading to wasted storage space.

# CHAPTER 3

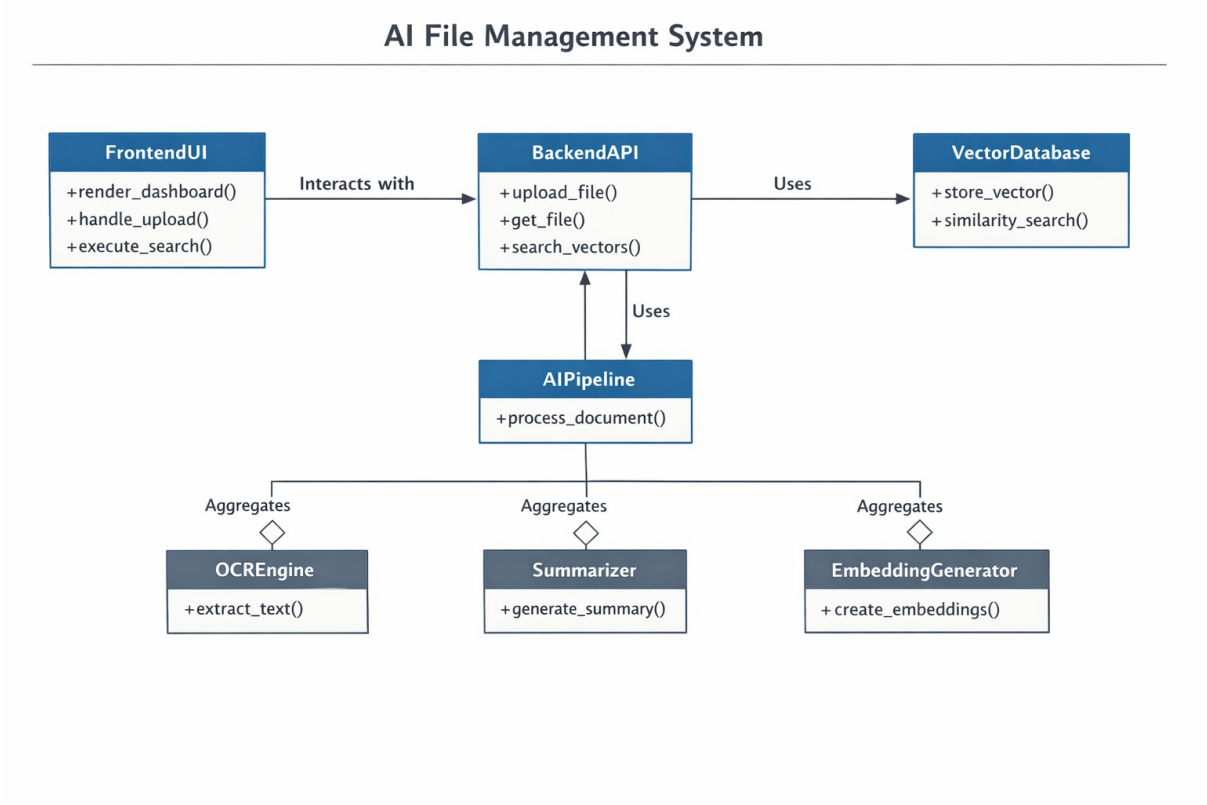
# SYSTEM REQUIREMENTS AND DESIGN OVERVIEW

## 3.1 FUNCTIONAL REQUIREMENTS

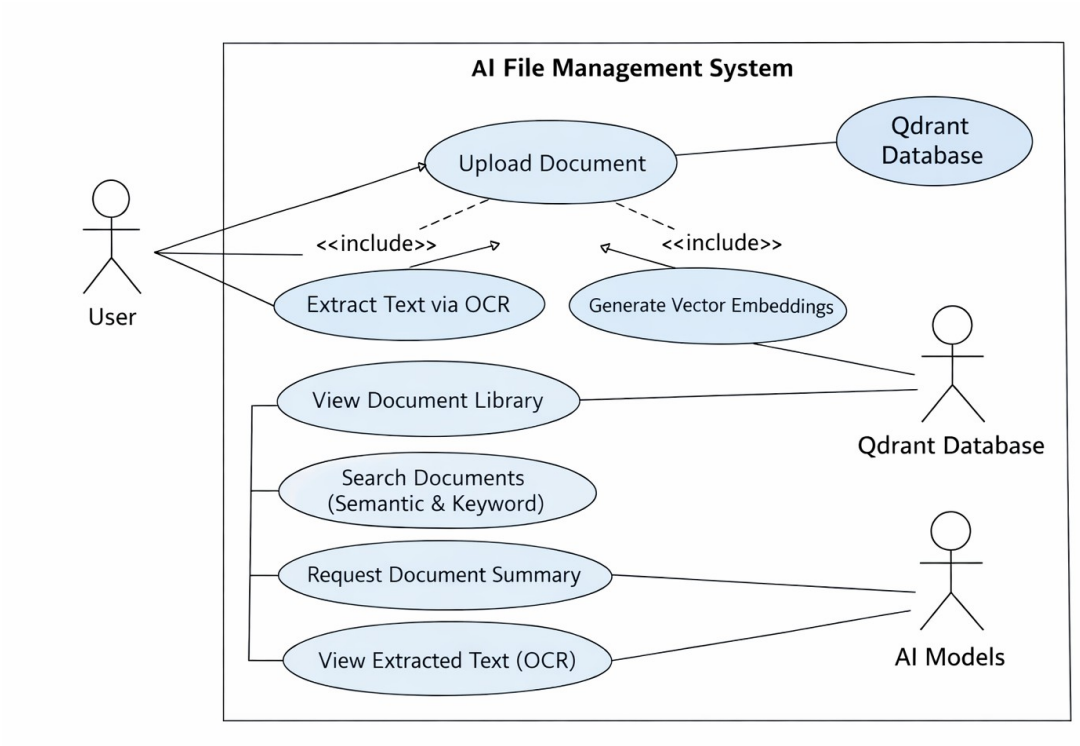
- **File Upload & Parsing:** The system must accept various file types (PDF, TXT, images) and safely store them.
- **Text Extraction (OCR):** The system must automatically trigger OCR sequences for image files to extract readable text.
- **Semantic Search:** Users must be able to search the database using natural language queries.
- **AI Summarization:** Users must be able to click a button to generate an AI-driven summary of any text-heavy document.
- **Deduplication:** The system must compute file hashes or embedding similarities to warn users when a duplicate file is uploaded.

## 3.2 SYSTEM DESIGN

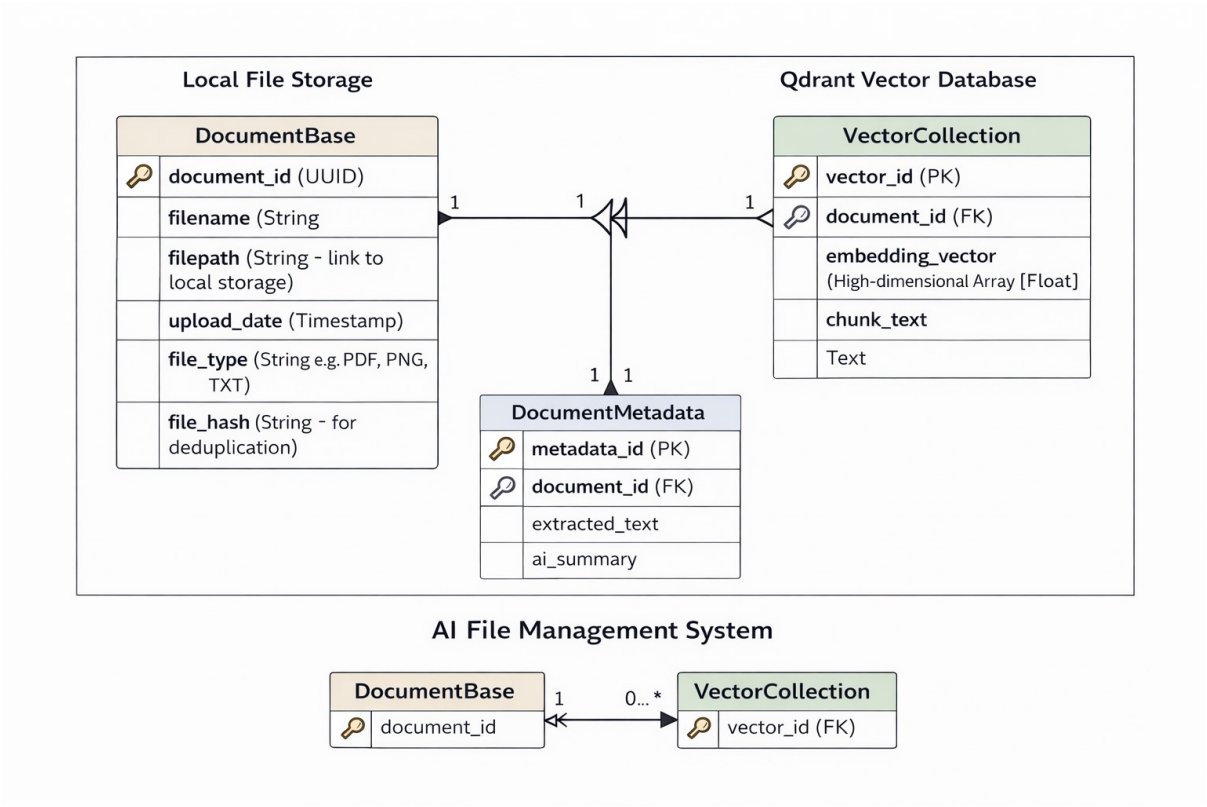
### 3.2.1 CLASS DIAGRAM



### 3.2.2 USE CASE DIAGRAM



### 3.2.3 DATABASE DESIGN



# CHAPTER 4

# IMPLEMENTATION

## 4.1 PROPOSED SYSTEM

### A) Technology Stack

- **Frontend:** Streamlit (Python-based framework for rapid UI development)
- **Backend:** FastAPI (High-performance asynchronous web framework)
- **Database:** Qdrant (Vector Database for semantic search), File System Storage
- **AI Inference:** Pre-trained NLP transformer models (for embeddings), NLP APIs (Groq/Local LLMs for summarization), and EasyOCR/Tesseract (for optical character recognition).

### B) Key Features & Implementation Details

Whenever a file is uploaded, the FastAPI backend intercepts it. If it is an image, the OCR module processes it to extract text. The text is then passed to an embedding model which converts paragraphs into numerical vectors. These vectors are indexed into Qdrant. When a user executes a search, their query is also vectorized, and Qdrant performs a nearest-neighbor search to return the most contextually relevant documents.

## 4.2 MODULES IN PROJECT

**Module 1: Document Processing & OCR**

This module handles the ingestion of files. It sanitizes file names, saves files securely, and detects the MIME type. If an image is detected, it passes the image matrix to the OCR engine, which identifies characters and outputs continuous text strings.

**Module 2: AI Summarization & Tagging**

This module takes the extracted text of a document and passes it to an AI summarization model. The model reads the context and generates a concise abstract of the file. Additionally, a tagging algorithm identifies key noun-phrases and automatically assigns metadata tags to the document to aid in future filtering.

**Module 3: Semantic Search and Deduplication**

The core intelligence of the application. Semantic search utilizes dense vector embeddings to match user queries with document content based on *meaning* rather than exact spelling. The deduplication logic runs upon upload, checking the cryptographic hash (or embedding similarity) of the new file against the existing database to prevent redundant storage.

# CHAPTER 5

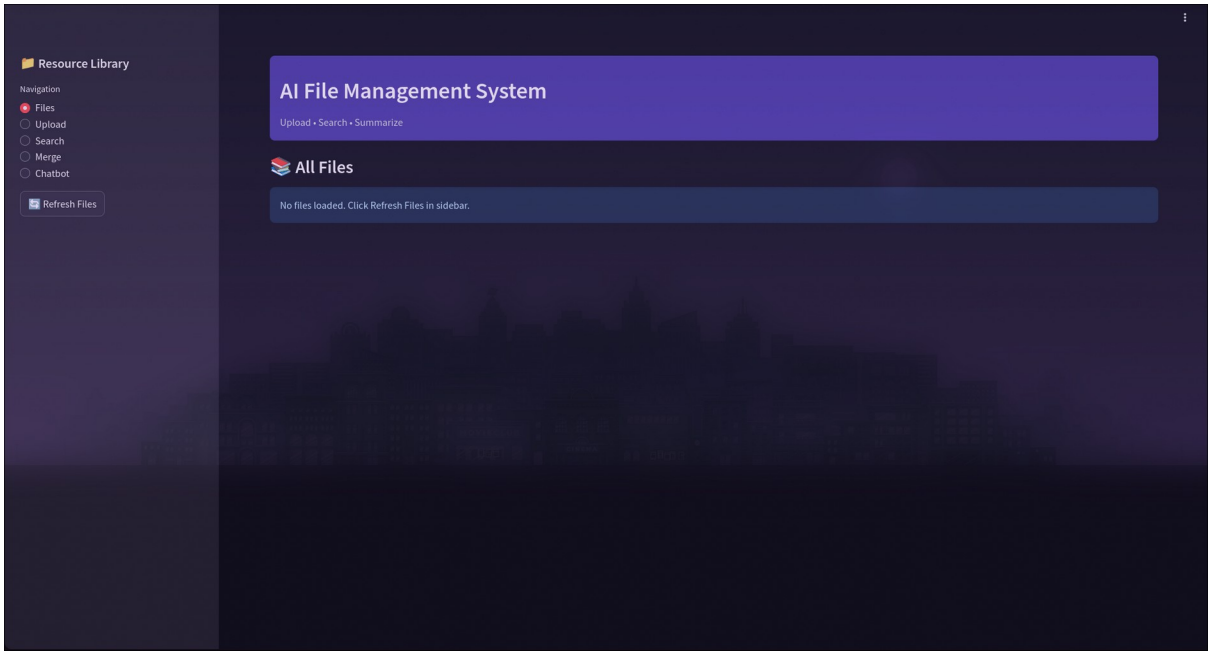
# TESTING AND RESULT

## 5.1 VALIDATION

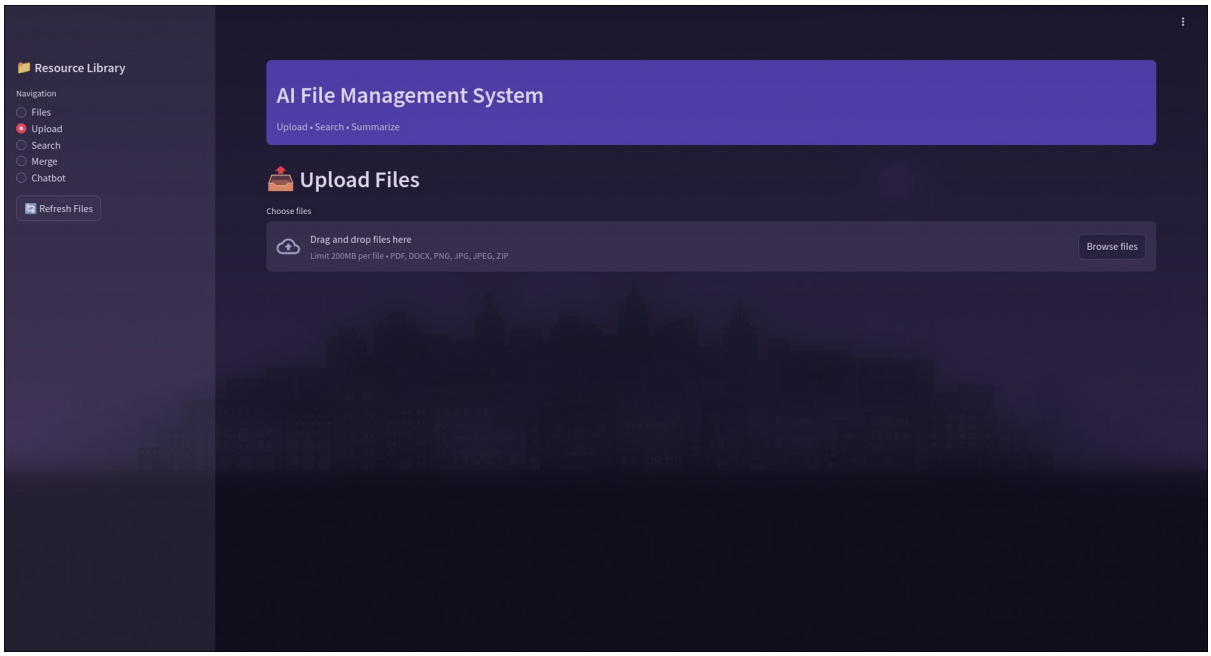
- **Unit Testing:** Individual functional scripts were tested. For example, test_ocr.py was executed to ensure the OCR engine accurately extracts text from varying image qualities without crashing.
- **Integration Testing:** Ensuring the Streamlit frontend properly communicates with the FastAPI backend, and that the backend successfully pushes and pulls vector data from the Qdrant database without timeout errors.
- **User Testing:** Testing the UI to ensure document upload progress bars accurately reflect state, and that search queries return within an acceptable timeframe (under 2 seconds).
- **Performance Testing:** Evaluating the vector database's retrieval speed as the number of uploaded documents scales, and monitoring the memory consumption of the local AI embedding models.

## 5.2 RESULTS

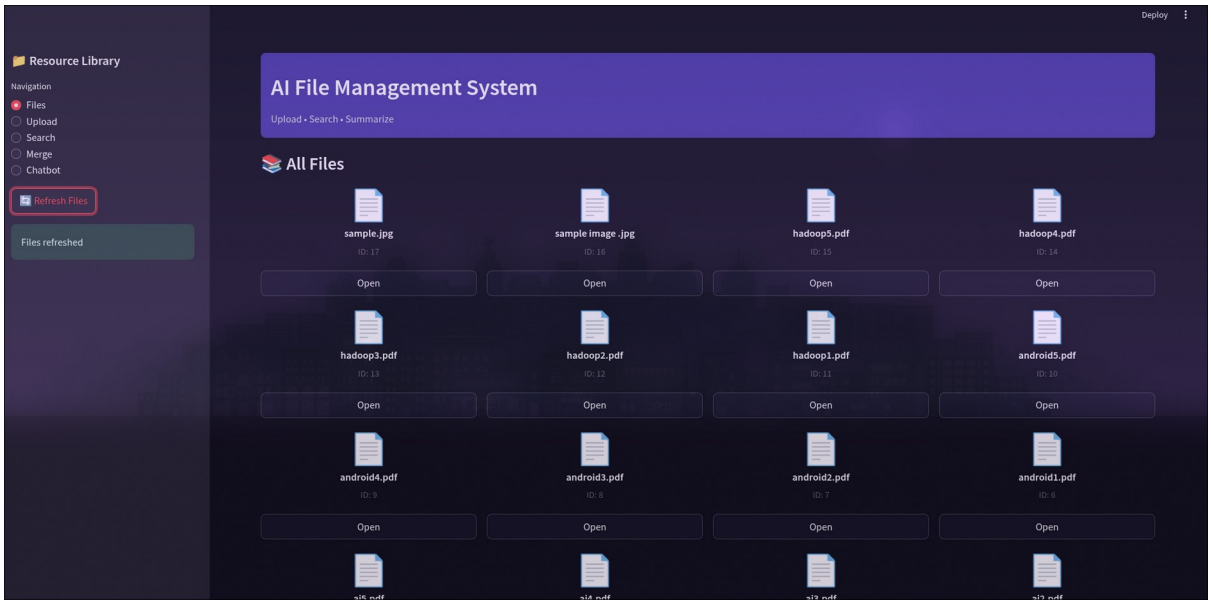
### Fig. 1: Dashboard Overview Panel




### Fig. 2: File Upload Interface with Drag-and-Drop



## Fig. 3: File Repository Listing




## Fig. 4: Semantic Search returning relevant context

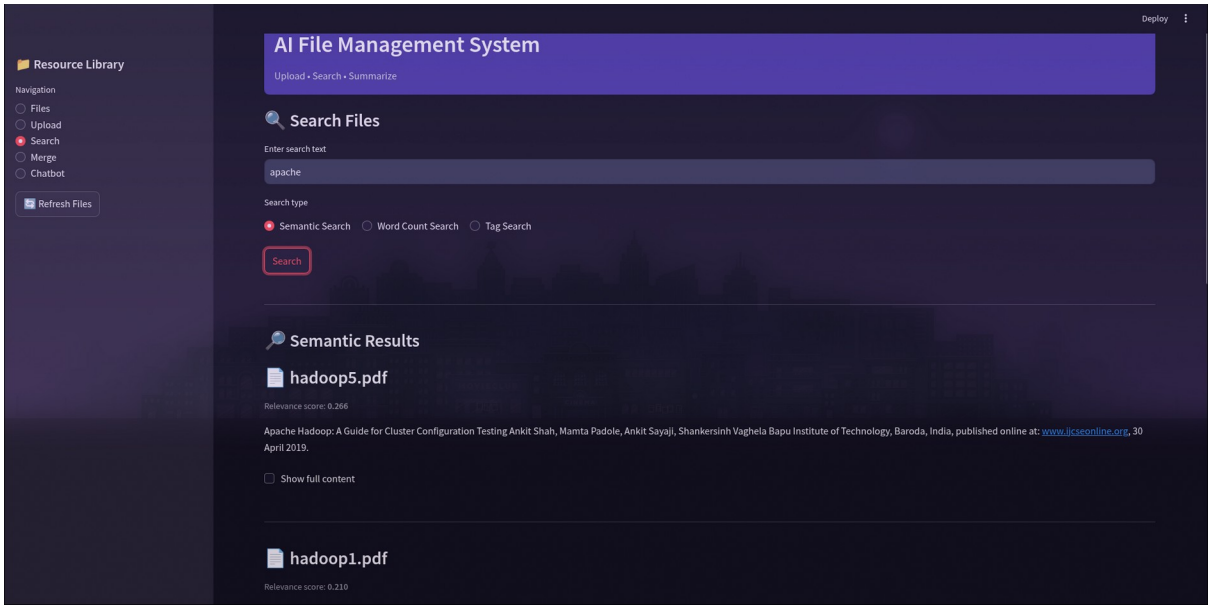


**Fig. 5: AI Generating a summary for a long PDF**

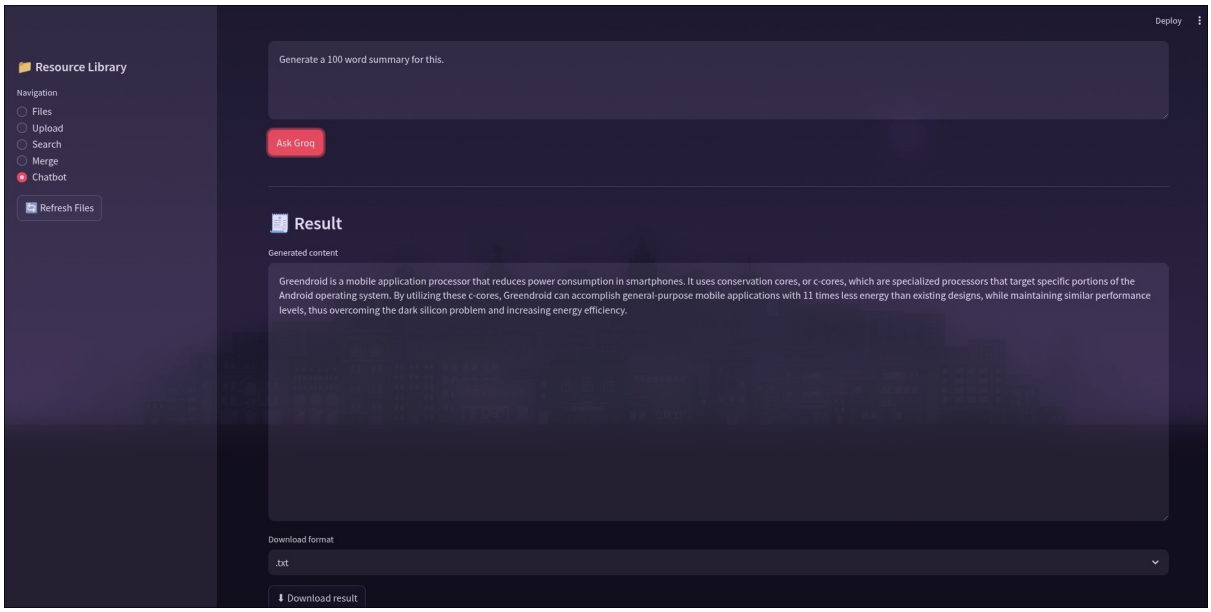
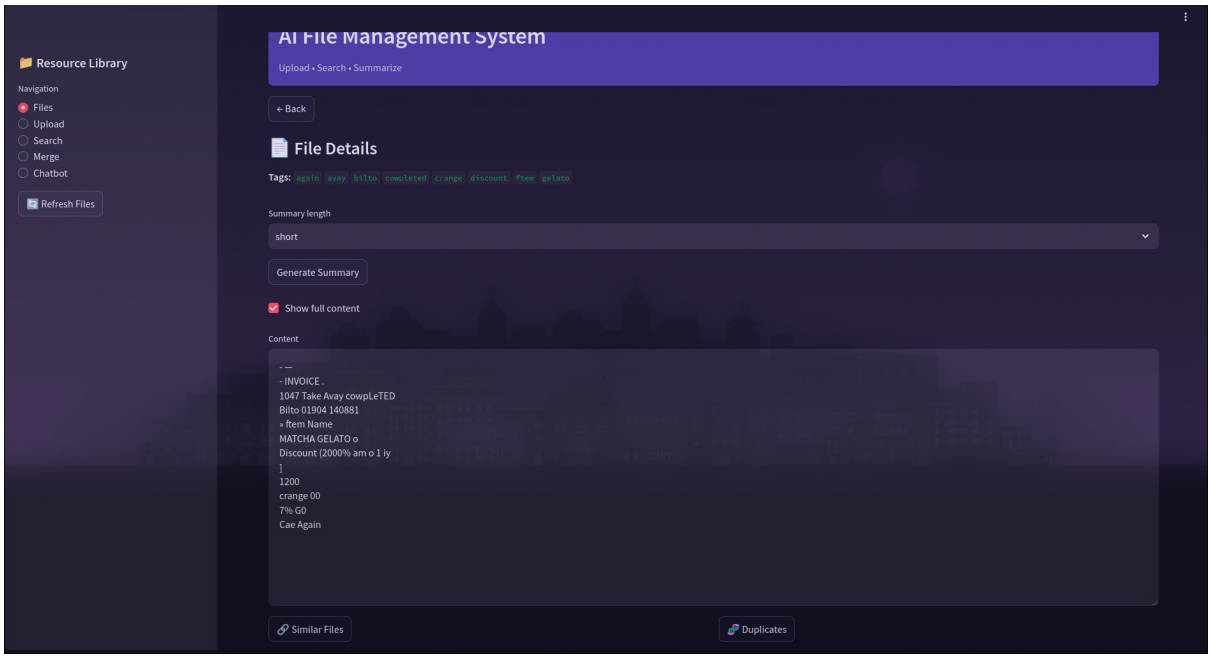



**Fig. 6: Successful text extraction from a scanned receipt**



# CHAPTER 6

## CONCLUSION

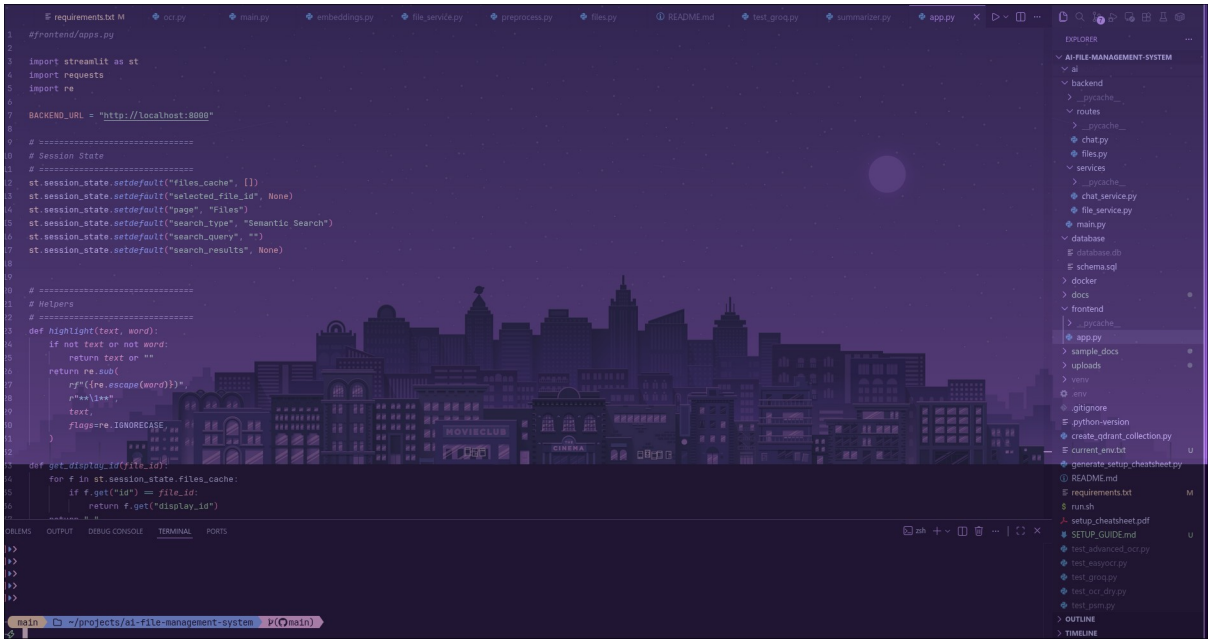
The AI File Management System successfully demonstrates the powerful application of artificial intelligence in everyday software utilities. By integrating FastAPI, Streamlit, and advanced machine learning models (OCR, Embeddings, LLM Summarization), the traditional, tedious process of file organization and retrieval has been heavily automated. The implementation of semantic search via Qdrant allows users to find documents based on contextual meaning, fundamentally upgrading the user experience. Future scope for this project includes implementing multi-user authentication, cloud-deployment scalability using Docker, and adding conversational AI agents to allow users to "chat" directly with their document repository.

# REFERENCES


1. **FastAPI Documentation:** https://fastapi.tiangolo.com/
2. **Streamlit Documentation:** https://docs.streamlit.io/
3. **Qdrant Vector Database:** https://qdrant.tech/documentation/
4. **EasyOCR Documentation:** https://jaided.ai/easyocr/
5. **Sentence Transformers (HuggingFace):** https://www.sbert.net/
6. "Attention Is All You Need" (Vaswani et al., 2017) - Foundational paper for modern NLP architectures utilized in summarization and embeddings.

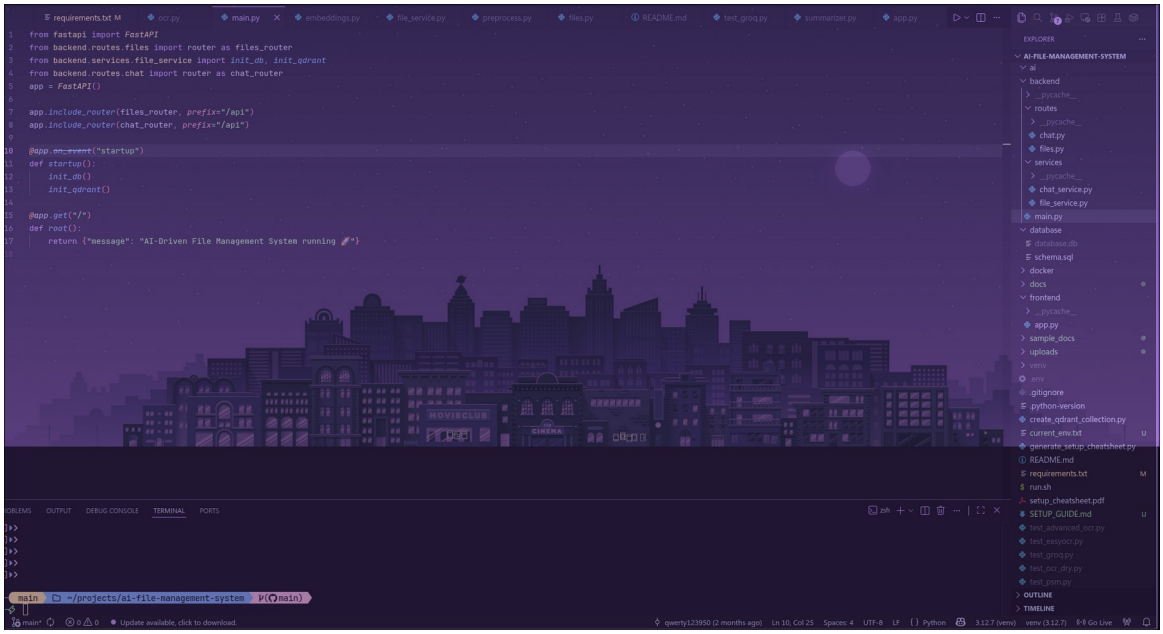
# APPENDIX - A: Source Code

## 1. app.py (Streamlit Frontend logic)



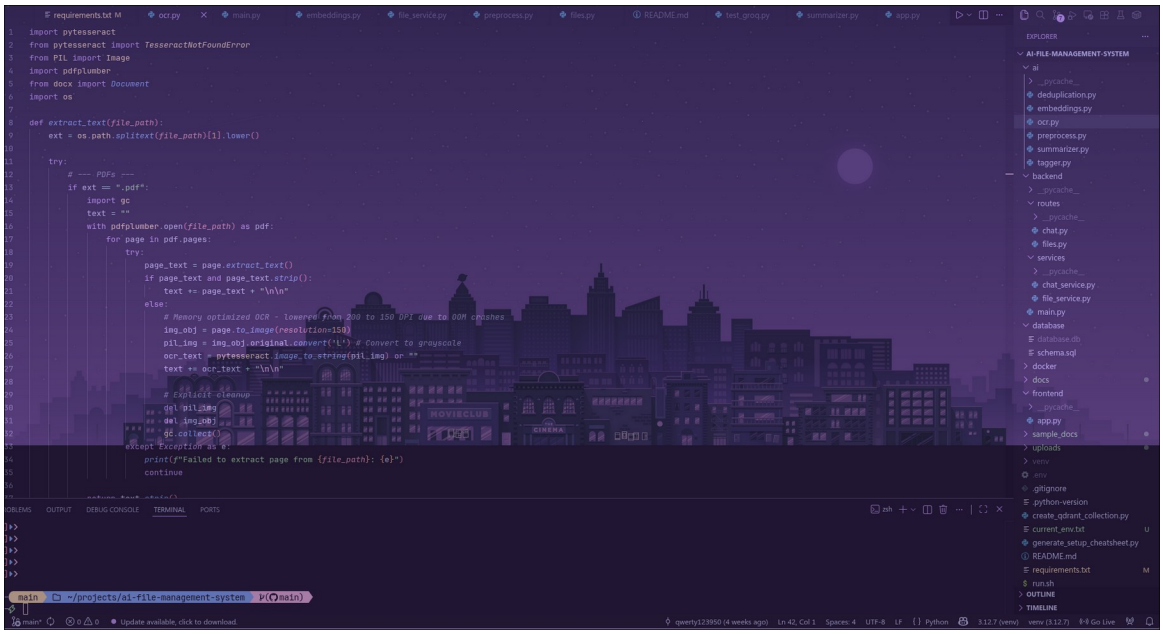
```
#frontend/apps.py

import streamlit as st
import requests
import re

BACKEND_URL = "http://localhost:8000"

# ==============================
# Session State
# ==============================
st.session_state.setdefault("files_cache", [])
st.session_state.setdefault("selected_file_id", None)
st.session_state.setdefault("page", "Files")
st.session_state.setdefault("search_type", "Semantic Search")
st.session_state.setdefault("search_query", "")
st.session_state.setdefault("search_results", None)


# ==============================
# Helpers
# ==============================
def highlight(text, word):
    if not text or not word:
        return text or ""
    return re.sub(
        rf"({re.escape(word)})",
        r"**\1**",
        text,
        flags=re.IGNORECASE
    )

def get_display_id(file_id):
    for f in st.session_state.files_cache:
        if f.get("id") == file_id:
            return f.get("display_id")
```

## 2. main.py (FastAPI backend routing)



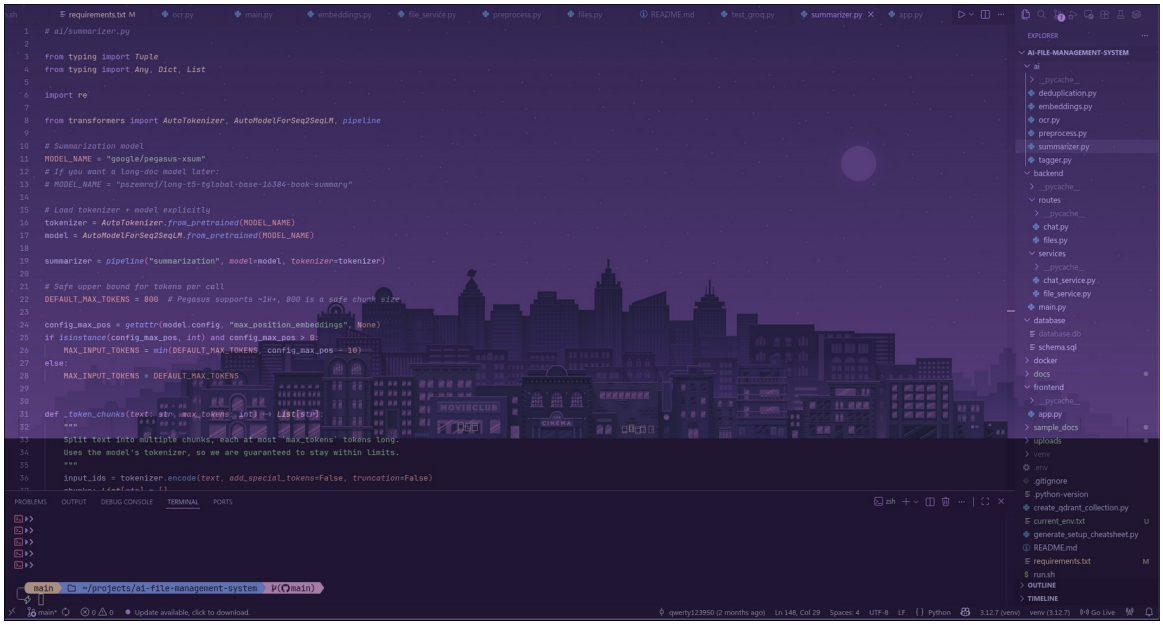
```
from fastapi import FastAPI
from backend.routes.files import router as files_router
from backend.services.file_service import init_db, init_qdrant
from backend.routes.chat import router as chat_router
app = FastAPI()

app.include_router(files_router, prefix="/api")
app.include_router(chat_router, prefix="/api")

@app.on_event("startup")
def startup():
    init_db()
    init_qdrant()

@app.get("/")
def root():
    return {"message": "AI-Driven File Management System running 🚀"}
```

## 3. ai/ocr.py (OCR processing logic)



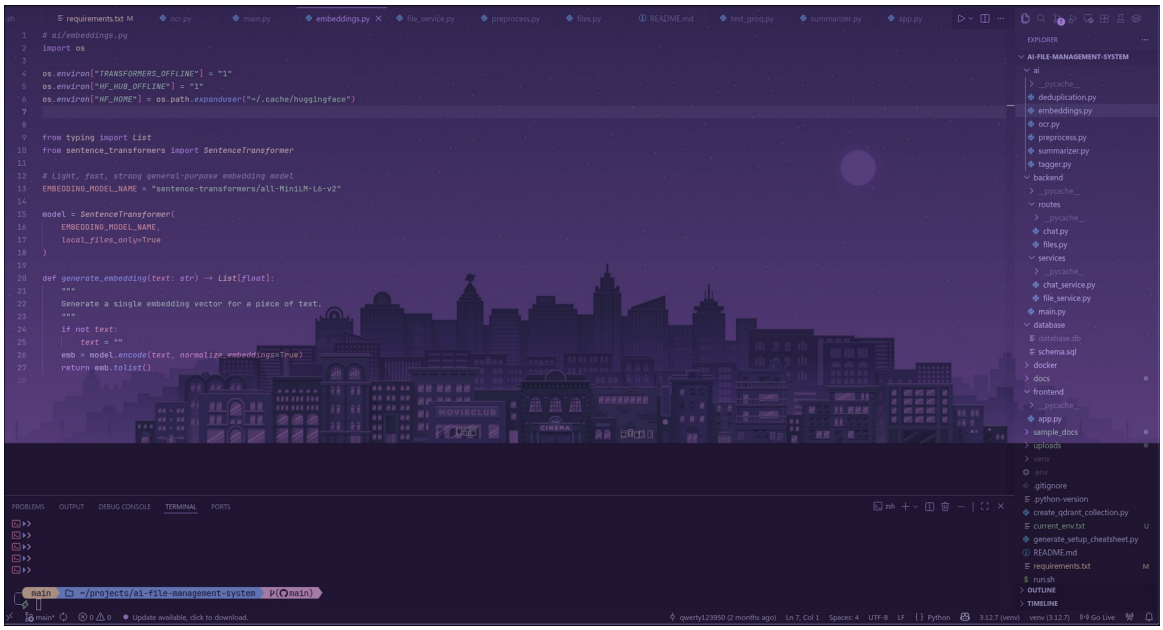
```
import pytesseract
from pytesseract import TesseractNotFoundError
from PIL import Image
import pdfplumber
from docx import Document
import os

def extract_text(file_path):
    ext = os.path.splitext(file_path)[1].lower()

    try:
        # --- PDFs ---
        if ext == ".pdf":
            import gc
            text = ""
            with pdfplumber.open(file_path) as pdf:
                for page in pdf.pages:
                    try:
                        page_text = page.extract_text()
                        if page_text and page_text.strip():
                            text += page_text + "\n\n"
                        else:
                            # Memory optimized OCR - lowered from 200 to 150 DPI due to OOM crashes
                            img_obj = page.to_image(resolution=150)
                            pil_img = img_obj.original.convert('L') # Convert to grayscale
                            ocr_text = pytesseract.image_to_string(pil_img) or ""
                            text += ocr_text + "\n\n"

                            # Explicit cleanup
                            del pil_img
                            del img_obj
                            gc.collect()
                    except Exception as e:
                        print(f"Failed to extract page from {file_path}: {e}")
                        continue
```

## 4. ai/summarizer.py (Summarization logic)



```
# ai/summarizer.py

from typing import Tuple
from typing import Any, Dict, List

import re

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline

# Summarization model
MODEL_NAME = "google/pegasus-xsum"
# If you want a long-doc model later:
# MODEL_NAME = "pszemraj/long-t5-tglobal-base-16384-book-summary"

# Load tokenizer + model explicitly
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)

summarizer = pipeline("summarization", model=model, tokenizer=tokenizer)

# Safe upper bound for tokens per call
DEFAULT_MAX_TOKENS = 800  # Pegasus supports ~1k+, 800 is a safe chunk size

config_max_pos = getattr(model.config, "max_position_embeddings", None)
if isinstance(config_max_pos, int) and config_max_pos > 0:
    MAX_INPUT_TOKENS = min(DEFAULT_MAX_TOKENS, config_max_pos - 10)
else:
    MAX_INPUT_TOKENS = DEFAULT_MAX_TOKENS


def _token_chunks(text: str, max_tokens: int) -> List[str]:
    """
    Split text into multiple chunks, each at most 'max_tokens' tokens long.
    Uses the model's tokenizer, so we are guaranteed to stay within limits.
    """
    input_ids = tokenizer.encode(text, add_special_tokens=False, truncation=False)
```

**5. ai/embeddings.py (Vector creation setup)**




# APPENDIX - B: User Manual

1. **Starting the System:** Run the backend server using the provided shell script (sh run.sh or uvicorn). Start the frontend using streamlit run app.py.
2. **Uploading a File:** Navigate to the 'Upload' tab, select your PDF or image, and click 'Upload'. Wait for the success notification.
3. **Searching:** Go to the 'Search' tab, type your natural language query into the search bar, and view the intelligently matched documents.
4. **Summarizing:** From the Document Library, select a file and click the "Generate Summary" button to view a quick synopsis of the document's contents.